



The Dark Side of Computing: Silent Data Corruptions

Dimitris Gizopoulos^{id}, University of Athens

Silent data corruptions due to defective silicon cause erroneous program results. Yet nobody knows how severe and frequent the problem is, how much we need to invest in solving it, and who should pay the bill.

Computing is unimaginably beautiful and powerful. In this essay, we touch on a problem that can undermine its beauty and might. We discuss what is probably the darkest side of computing: *silicon defects that corrupt our programs' data and we don't know it.*

Both the beauty and the might of computing result from two properties that all machine users demand.

Property #1: the computed result is *correct*. Program results are used to take important decisions and correctness is nonnegotiable. Depending on the domain, “correctness” may differ. The only correct answer may be “9,753,” while in some cases any answer between “9,500” and “9,800” is equally correct.

Property #2: the computation is *fast*. Nobody needs a correct result if it comes late. The result must reach the user in milliseconds, seconds, an hour, or two weeks. In domains with strict timing requirements, speed means meeting deadlines or lives, property, money is lost. In other domains speed means “as fast as possible” or “faster than the previous product.”

Humankind never stops pushing the limits of computing engines and always demands correct and fast computations. Often, computer designers must decide which of the two matters more—the choice depends on the criticality of the applications running on the system. Thus, as in all engineering disciplines, *cost matters*. Neither correctness nor speed comes for free. Both incur dramatic costs during design, manufacturing, and operation.

Correctness of operation is constantly in danger when we push the limits: from materials imperfections and flaws of the chip, board, system manufacturing, through the network errors, all of the way to programmer errors and arithmetic precision in parallel programs. Dealing with the aforementioned requires extra time and resources for correctness. For silicon chips, correctness is jeopardized at

design time (hardware bugs), manufacturing time (silicon defects, variability of physical properties), and mission time (aging, radiation).

In an ideal world, the manufacturers of every part and the assembled whole of a computer would invest resources to deliver flawless products; this never happens. Verification of a new design, validation of few prototypes, and comprehensive testing of every chip, board, and system, have tight time and cost bounds. With products delivered in just six months or one year, some chips in the manufactured quantities will inevitably contain defects that are not screened at production. Even the most diligent manufacturing testing process can detect between 95% and 99% of the modeled defects. But even if we were able to screen every defective chip at production, there are new defects that can appear during chip lifetime due to aging or operating conditions.

SILICON DEFECTS AFFECTING PROGRAMS

What is a silicon defect and how it can affect our programs?

Let's take a simple example. Consider a two-input logic OR gate inside an integer multiplier of your CPU or GPU silicon chip. Due to imperfect manufacturing (a broken wire or a short circuit) this gate occasionally gives the wrong logic OR result when both inputs are at logic 0 (the gate output is 1 instead of the correct 0); the other three input combinations give the correct result 1. This defect, which may be either persistent or may depend on certain rare

physical parameters, will affect our program execution when *all of the following conditions are valid*:

1. Our program uses multiplication instructions—the program used the defective arithmetic unit.
2. The operands of some of the multiplications feed the OR gate with both inputs at logic 0—the defective gate will produce the wrong result.
3. There is at least one path of logic gates from the defective gate to the multiplier outputs that propagates the wrong gate result—there is at least one wrong multiplication result.
4. The wrong multiplication result propagates through subsequent instructions in a way that harms execution: the program hangs, crashes, or raises an exception because the wrong multiplication result affects the control flow and changes the values of pointers to memory or ends with an incorrect result because the wrong multiplication affects the data flow.

When any of the conditions are not met, the defective OR gate does not matter for the program. [Figure 1](#) visualizes the conditions, showing how a defective gate in a multiplier produces a wrong result which propagates through to erroneous program output, recognizing a dog instead of a cat.

Similarly to the multiplier, every single hardware unit inside a silicon chip of a CPU, GPU, or domain-specific

accelerator (dynamic random-access memory chips too) can contain defects: registers, caches, buffers, control logic, and arithmetic units. Numbers manipulated by, stored in, and transferred through hardware units may be wrong due to silicon defects.

VISIBLE AND SILENT EFFECTS OF DEFECTIVE SILICON

When chips contain defects in their silicon that affect our programs, the first critical question is: “do we know the program ran incorrectly?” If defective silicon leads to a program generating an exception or a system crash (because hardware or software caught the erroneous operation), although we don't like it, at least we know it happened. We try again (hoping that erroneous execution was temporary), and if it happens again, we don't trust the computer for our subsequent work, and we either fix it or buy a new one. But such “visible” errors are not always the case.

Invisible errors [called *silent errors* or *silent data corruptions* (SDCs)] happen when a program runs on a defective chip, completes its execution, does not crash the process or the system, does not raise any exception, and does not produce an output that is obviously wrong. Still the program's result is incorrect and nobody knows! No hardware or software detection mechanisms caught it.

Many readers will, impulsively, think “then add hardware or software detection mechanisms!” Correct, but detection is very costly, and we need to think twice before implementing it. For example, hardware-based error detection and correction codes for memories can incur anything between 2% and 125% (!) extra silicon and power, while software-based redundant execution (double or triple redundancy) has a 100% or 200% extra performance and energy cost. Who is ready to take this?

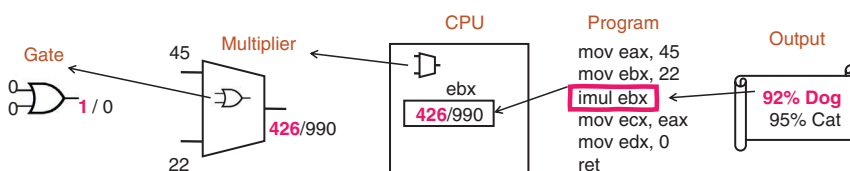
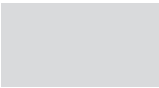


FIGURE 1. From a silicon gate to the program output. The defective gate must receive the inputs that give the wrong output *plus* the gate's output must propagate to the multiplier output *plus* the program must contain a multiplication instruction *plus* the wrong multiplication must affect the program output.



panelists were wondering if it exists or if it is a myth.¹ For decades, we were convinced that SDCs were a myth, or, in the worst-case scenario, an SDC might affect the unlucky user of one in a million (or billion) computers. The reality is different according to the major users of computing chips of our days: data center operating companies.

Hyperscalers (Meta,² Google,³ Alibaba⁴) have disclosed over the last few years an unexpectedly high number of CPUs (1 in 1000) that lead to SDCs—program executions that produce wrong results without any observable indication. Other types of processing chips [artificial intelligence (AI) accelerators from Google⁵ and Meta,⁶ and NVIDIA GPUs⁷] are also reported to generate SDCs. Reports agree that the root cause of such SDCs are chips which are born defective (escaped manufacturing testing), become defective (aging), or just differ from each other (variability).

A short note on terminology is important here. SDCs are neither detected nor corrected. Once the effect of a defect is detected, it is not silent anymore. An SDC is an eventual (terrible) result of a defect when a program runs. It is not detected; that's why it is called silent (unlike effects like crashes which can't be missed). Per the fault-tolerant computing terminology used for decades: We detect defects (or their models: the faults) and we correct their effects—the errors—at different abstraction layers (circuit, microarchitecture, software) so that the machine delivers the expected service. Public statements like “an SDC detection scheme” and “the SDC is corrected” (even if the speaker or author does

know what they talk about) don't educate newcomers to the topic.

What the computing community needs to do for SDCs is to minimize or, ideally, zero their rate. In a perfect world, the result of a computation produced by chips should be either 1) correct (defect does not matter or correction worked), or 2) incorrect but

- Which instructions are more likely to lead to SDCs?
- Which silicon technology nodes are more likely to generate SDCs?

Without knowing which hardware units are likely to generate SDCs, how can one protect them? Without knowing which instruction classes are



For decades, we were convinced that SDCs were a myth, or, in the worst-case scenario, an SDC might affect the unlucky user of one in a million (or billion) computers.

known to be so (detection worked but no correction is possible). A perfect “zero-SDCs” world is impossible because it requires unaffordable protection costs.

WHAT COMPANIES REALLY DISCLOSED?

Despite the hype of the incident's reports, it is still unclear how severe the problem is and what its specifics are. Table 1 summarizes what hyperscalers reported.

Clearly, there are still many open questions, such as follows:

- What are the real SDC rates, that is, corrupted program outputs over time when real programs run?
- Which chip microarchitectures are more likely to generate SDCs?
- Which hardware units (sizes, counts, designs) are more likely to generate SDCs when defective (Alibaba hinted floating-point and vector hardware units)?

mainly affected by defects, how can one harden them? Without knowing which CPU microarchitectures are more likely to generate SDCs, how can a hyperscaler decide which to purchase, and how can a chip vendor improve future designs?

Most importantly, how can anyone (chips vendor, system integrator, hyperscaler) take decisions for protection (and pay the costs) against SDCs if we don't know the *scale and severity of the problem*? Bottomline: How many SDCs are happening out there? How many programs per unit of time (hour, day) complete execution with the wrong output in a data center and nobody knows?

Thinking more carefully, the question does not make sense! “Measure the number of SDCs” is a contradiction in terms. We can't measure something that we can't observe because it is silent. So, essentially the mandate changes to: “Estimate the number of SDCs because you can never measure them.”

The answer to the estimation request depends on many aspects of

TABLE 1. Hyperscalers disclosures.

	Meta	Google	Alibaba
Text about the number of CPUs that produce SDCs	“... hundreds of CPUs across hundreds of thousands of machines ...”	“... a few mercurial cores per several thousand machines ...”	“... 3.61% of the CPUs are identified to cause SDCs ...”
Interpretation of the text in defective parts (CPUs) per million (DPPM)	≈ 1000	< 1000	= 361

large-scale systems: their size, the defectivity of the chips, the workload types, the execution times, and so on. A first-order estimation (that employs microarchitecture-level fault injection in x86 CPU models) about the number of SDCs considering defects in the arithmetic units (scalar and vector, integer, and floating-point) of data center CPUs, underlines the urgency of the matter and calls for action.⁸

Figure 2 shows some examples of this first-order estimation for varying data center size (number of cores), workload execution time (minutes), and chips defectivity [defective parts per million (DPPM)]. It shows the estimated number of SDCs generated per day in the data center.

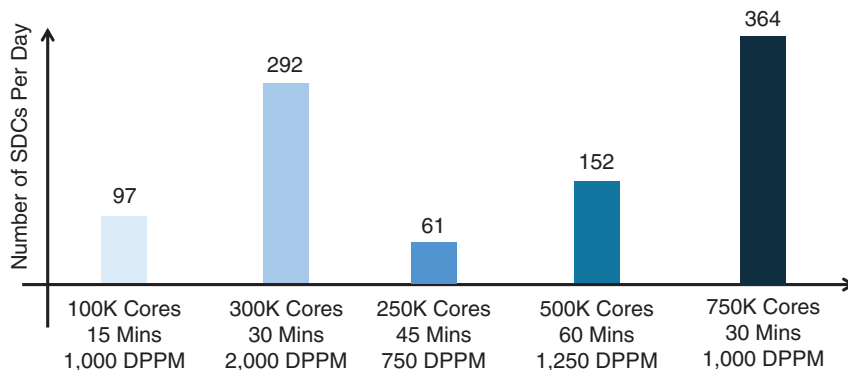


FIGURE 2. First-order estimation using microarchitectural simulation for the rate of SDCs due to defects in arithmetic units of data center x86 CPUs. Bars correspond to different data center size, workload time, and defectivity. These rates can't be measured in real production systems since the effect of the defects is silent.

The number of incidents in Figure 2 do not refer to visible effects of CPUs defects but to silent ones. When a CPU in a data center causes crashes (or any visible effect), it is removed from the fleet and does not cause crashes anymore. The fleet of CPUs produces hundreds of erroneous program outputs delivered to the users that have absolutely no idea! Now, try to answer: How many SDCs can we afford? Zero! Let's see what we can do to reach this ideal.

REDUCING SDCs AND THE COST

There are multiple ways to reduce the rate of SDCs. Table 2 summarizes them and the corresponding costs.

Everything costs; who is going to pay?

MORE DISCLOSURES FROM INDUSTRY?

We must praise Meta for the first SDCs disclosure and subsequent action to engage with academia⁹ and Google and Alibaba for confirming. We must also praise AMD, Intel, NVIDIA, and Arm for joining hyperscalers in the Open Compute Project (OCP)¹⁰ and helping awareness about the problem and engagement with universities.¹¹ Initial disclosures were followed by professional organization initiatives (the IEEE Computer Society RAS in the Datacenter Summit 2024¹²) and panels,¹³ keynotes,¹⁴ special sessions,^{15,16} publications,^{8,17} and blog posts.^{18,19}

However, beyond the current awareness state, it is unlikely that companies will provide more detailed, thus useful, information about defective silicon chips and the SDC rates they generate. Two reasons easily explain this pessimistic expectation.

First, companies make their profit out of their product features, the properties that the public perceive as good! Performance comes first; power and energy consumption are also important; reliability (reduced failure rates) is rarely (never!) a selling point. The fear that a machine may fail even once every many months, or, even worse, it may produce an unnoticed corrupted output, is terrifying. Nobody talks about the errors their system generates.

TABLE 2. Reducing the rate of SDCs and associated costs.

Approach	Why it reduces SDCs	Why it adds cost
Better manufacturing testing	Fewer manufacturing defects will escape into production. Fewer SDCs!	High volume manufacturing test time is expensive; chips cost more.
Reduced chip variability	Marginally nominal chips are not deployed in the systems. Fewer SDCs!	The production yield is reduced; chips cost more.
Enhanced hardware defect tolerance	Mission mode defects (due to radiation and aging for example) are detected or corrected at the hardware level. Fewer SDCs!	Every hardware redundancy scheme comes with design, verification, area, power, performance overheads; system costs more.
Enhanced software error tolerance	Erroneous calculations are detected and corrected. Fewer SDCs!	Every software redundancy scheme comes with performance and energy overheads; system costs more.

Second, failure rates (including SDC rates) are very difficult to define, measure, and reproduce. When a company makes claims about its product rating in speed or power, it is reasonably possible to reproduce the setup and verify that the claim is accurate. For failure rates (or SDC rates) there is no practical way to reproduce any claim: If a company states that its chip generates one silent data corruption per year in a population of 1,000,000 chips, how can one verify the statement?

SDC-AWARENESS IN COST MODELS

The recent rise of awareness about the existence of SDCs in cloud computing is very important. Hyperscalers now require better quality chips from silicon vendors while also focusing on detecting defective chips in fleets. The entire industry is looking for solutions.

It is probably time for hyperscalers to take the rates of SDCs into consideration in cost models. A data center user may occasionally receive erroneous execution outputs when their programs run in the cloud. For a potentially “high SDC rate” data center rental time, users should deal with the integrity of the program results themselves (and thus pay less). On the other hand, a guaranteed “low SDC rate” data center time means that the provider takes care of the correctness of the delivered results and, for this reason, can charge more.

It is again a matter of cost and who takes it. It can be the chip vendor, it can be the hyperscaler, it can be the user/customer. SDC rates should be added to the cost equation of data centers on both the operator side and the user side.

SDCS FROM DATA PARALLEL ARCHITECTURES

This short essay mainly focuses on SDCs from CPU chips because disclosures mainly refer to them. But who expects things to be better in massive data-parallel architectures like a GPU

or a programmable AI accelerator?^{20,21} The data-centric architecture of such chips and their massive deployment for machine learning/AI systems intuitively points to huge rates of SDCs. The discussion of the text so far and

but continuous screening in the field can catch complex real-world SDC scenarios that the short, focused manufacturing testing process can't. After all, in data centers, what matters is the

Tackling the challenge of SDCs requires contributions from researchers and practitioners across the entire computing systems stack.

the research directions that conclude the essay apply to all different chip types. If a CPU contains a multiplier like the one discussed previously, how many multipliers does a GPU, or a custom AI accelerator, have?

PRACTICAL (COST-EFFECTIVE) RESEARCH DIRECTIONS

Dependable computing has always been an exciting field. In times that technology is pushed to its limits, statistics are not on our side. The scale of computing systems of our days and our greed for speed challenges our ability to guarantee correctness of computers operations. A (nonexhaustive) list of practical (cost-effective) research directions that can contribute to our quest for the holy grail of zero-SDC systems includes:

- › *Estimations for the real SDC rates (not just the number of chips that occasionally produce SDCs):* The single defective chip in a thousand may generate zero SDCs if the defective unit is not used; it can generate millions of SDCs per day if workloads use it massively. Ideally, the SDC rate estimations must correlate them to instruction or workload classes to assist software-based tolerance.
- › *Effective periodic scanning of data centers fleets:* Manufacturing testing should constantly improve to screen defective parts,

detection of defective chips and their replacement.

- › *Tolerance at the hardware and the software layers:* Tolerance refers to approaches that aim at continuing the system operation despite the existence of defects. In the case of SDCs in large computing infrastructures where massive redundancy (the common denominator of all tolerance approaches) is rarely affordable, tolerance is not a very intuitive option. When a data center operator knows a chip is defective, the only responsible action is to replace it. The only meaningful alternative would be the degraded operation of a chip with a known and contained defect in a hardware unit which is not used by workloads.

Tackling the challenge of SDCs requires contributions from researchers and practitioners across the entire computing systems stack: physical design, logic and microarchitecture design, instruction set architectures, systems software, compilers, and application software. While we certainly live in the “Golden Age of Computer Architecture” per John Hennessy’s and David Patterson’s words,²² we also live in the “Golden Age of Dependable Computing” and we all need to make sure computing remains as beautiful and as powerful as it should be. ■

ACKNOWLEDGMENTS

Thanks to the Computing Architectures column editors for the invitation to host the article in *Computer*, and all the team members of the Computer Architecture Lab of the University of Athens that work with me in the exciting research area. Research is supported by Meta, AMD, and the OCP.

REFERENCES

1. C. Constantinescu, I. Parulkar, R. Harper, and S. Michalak, "Silent data corruption — Myth or reality?" in *Proc. IEEE Int. Conf. Dependable Syst. Netw. (DSN)*, Anchorage, AK, USA, 2008, pp. 108–109, doi: [10.1109/DSN.2008.4630077](https://doi.org/10.1109/DSN.2008.4630077).
2. H. D. Dixit et al., "Silent data corruptions at scale," 2021, *arXiv:2102.11245*.
3. P. H. Hochschild et al., "Cores that don't count," in *Proc. Workshop Hot Topics Operating Syst. (HotOS)*, New York, NY, USA: ACM, 2021, pp. 9–16, doi: [10.1145/3458336.3465297](https://doi.org/10.1145/3458336.3465297).
4. S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo, "Understanding silent data corruptions in a large production CPU population," in *Proc. 29th Symp. Operating Syst. Princ. (SOSP)*, 2023, pp. 216–230, doi: [10.1145/3600006.3613149](https://doi.org/10.1145/3600006.3613149).
5. Y. He, M. Hutton, S. Chan, R. De Gruijl, R. Govindaraju, and N. Patil, "Understanding and mitigating hardware failures in deep learning training systems," in *Proc. 50th Int. Symp. Comput. Archit. (ISCA)*, 2023, pp. 1–16, doi: [10.1145/3579371.3589105](https://doi.org/10.1145/3579371.3589105).
6. D. Ma et al., "Dr. DNA: Combating silent data corruptions in deep learning using distribution of neuron activations," in *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2024, pp. 239–252, doi: [10.1145/3620666.3651349](https://doi.org/10.1145/3620666.3651349).
7. "Data center documentation – Version 535.129.03(Linux)/537.70(Windows)," NVIDIA Docs. Accessed: Apr. 4, 2024. [Online]. Available: <https://docs.nvidia.com/datacenter/tesla/tesla-release-notes-535-129-03/index.html>
8. O. Chatzopoulos, N. Karystinos, G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Veritas – Demystifying silent data corruptions: μ Arch-level modeling and fleet data of modern x86 CPUs," in *Proc. IEEE Int. Symp. High-Perf. Comput. Archit. (HPCA)*, 2025, pp. 1–14.
9. "Announcing the winners of the 2022 silent data corruptions at scale request for proposals," *Meta Research*, Apr. 4, 2025. [Online]. Available: <https://research.facebook.com/blog/2022/6/announcing-the-winners-of-the-2022-silent-data-corruptions-at-scale-request-for-proposals/>
10. "OCP's server resilience initiative: SDC academic research awards announced!" *Open Compute Project*, Jun. 4, 2024. [Online]. Available: <https://www.opencompute.org/blog/ocps-server-resilience-initiative-sdc-academic-research-awards-announced>
11. "Sudhanva Gurumurthi's post." LinkedIn. Accessed: Apr. 4, 2024. [Online]. Available: <https://www.linkedin.com/posts/sudhanva-gurumurthi-emerging-fault-modes-challenges-and-research-activity-7086699729228607490-Yv1z/>
12. "1st IEEE RAS in data centers summit." IEEE RAS 2024. Accessed: Apr. 4, 2024. [Online]. Available: <https://ieeeras.conferences.computer.org/2024/>
13. ACM SIGARCH. ISCA'22 - Lunch and Panel on Silent Data Corruptions. (Jul. 13, 2022). Accessed: Apr. 4, 2024. [Online Video]. <https://www.youtube.com/watch?v=j1IvGZZSm3k>
14. S. Hesley. ITC Wednesday Keynote. (Dec. 15, 2024). Accessed: Apr. 4, 2024. [Online Video]. <https://www.youtube.com/watch?v=aKnUAoCPkbE>
15. G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Silent data corruptions: The stealthy saboteurs of digital integrity," in *Proc. IEEE 29th Int. Symp. On-Line Testing Robust Syst. Des. (IOLTS)*, Crete, Greece, 2023, pp. 1–7, doi: [10.1109/IOLTS59296.2023.10224870](https://doi.org/10.1109/IOLTS59296.2023.10224870).
16. T. Macieira, S. Gurumurthy, S. Gurumurthi, A. Haggag, G. Papadimitriou, and D. Gizopoulos, "Silent data corruptions in computing: understand and quantify," in *Proc. IEEE 30th Int. Symp. On-Line Testing Robust Syst. Des. (IOLTS)*, Rennes, France, 2024, pp. 1–7, doi: [10.1109/IOLTS60994.2024.10616056](https://doi.org/10.1109/IOLTS60994.2024.10616056).
17. N. Karystinos, O. Chatzopoulos, G.-M. Fragkoulis, G. Papadimitriou, D. Gizopoulos, and S. Gurumurthi, "Harpocrates: breaking the silence of CPU faults through hardware-in-the-loop program generation," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Buenos Aires, Argentina, 2024, pp. 516–531, doi: [10.1109/ISCA59077.2024.00045](https://doi.org/10.1109/ISCA59077.2024.00045).
18. S. Gurumurthi, V. Sridharan, and S. Gurumurthy, "Emerging fault modes: Challenges and research opportunities," *ACM SIGARCH*, Jul. 17, 2023. [Online]. Available: <https://www.sigarch.org/emerging-fault-modes-challenges-and-research-opportunities/>
19. D. Gizopoulos, "SDCs: A B C," *ACM SIGARCH*, Sep. 16, 2024. [Online]. Available: <https://www.sigarch.org/sdcs-a-b-c/>
20. A. Vahdat and M. Lohmeyer, "Enabling next-generation AI workloads: Announcing TPU v5p and AI hypercomputer," *Google Cloud*, Dec. 7, 2023. [Online]. Available: <https://cloud.google.com/blog/products/ai-machine-learning/introducing-cloud-tpu-v5p-and-ai-hypercomputer>
21. "MTIA v1: Meta's first-generation AI inference accelerator," *AI at Meta*, May 18, 2023. [Online]. Available: <https://ai.meta.com/blog/meta-training-inference-accelerator-AI-MTIA>
22. J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, pp. 48–60, doi: [10.1145/3282307](https://doi.org/10.1145/3282307).

DIMITRIS GIZOPOULOS is a professor at the Department of Informatics and Telecommunications, University of Athens, 15784 Athens, Greece. Contact him at dgizop@di.uoa.gr.